

TEMA 5

PROCESOS Y ATRIBUTOS



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL

Tema 5. Procesos y atributos

❖ Objetivos

- Comprender el concepto de ejecución secuencial.
- Comprender la estructura y utilidad de un statement process.
- Analizar la sintaxis y funcionamiento de los distintos statements secuenciales de VHDL.
- Comprender los casos en los que debe emplearse una señal o una variable en un process.
- Analizar el formato más empleado en la síntesis de sistemas combinacionales y secuenciales.
- Exponer ejemplos de descripción de sistemas combinacionales y secuenciales mediante procesos.
- Analizar los atributos más usuales en la síntesis

Tema 5. Procesos y atributos

❖ Bibliografía

- John F. Wakerly. *Diseño Digital. Principios y Prácticas*. Ed. Prentice Hall. 2001.
- William Kafig. *VHDL 101. Everything you need to know to get started*. Ed. Newnes. 2011.
- Gina R. Smith. *FPGA 101. Everything you need to know to get started*. Ed. Newnes. 2010.
- Pong P. Chu. *FPGA Prototyping by VHDL Examples*. Ed. Wiley. 2008.
- Ion Grout. *Digital Systems Design with FPGAs and CPLDs*. Ed Elsevier Ltd. 2008.
- Peter Wilson. *Design Recipes for FPGAs*. Ed. Newnes. 2007.
- David Pellerin y otros. *VHDL Made Easy!*. Ed. Mc Graw Hill. 1997.
- P. J. Ashenden. *The Student's Guide to VHDL*. Ed. Morgan Kaufmann. 2008.
- Peter J. Ashenden. *The Designer's Guide to VHDL*. Ed. Elsevier Inc. 2008.
- Douglas Perry. *VHDL*. Ed. Mc-Graw Hill. 1998.
- R. C. Cofer and Benjamin F. Harding. *Rapid System Prototyping with FPGAs*. Ed. Elsevier Inc. 2006.



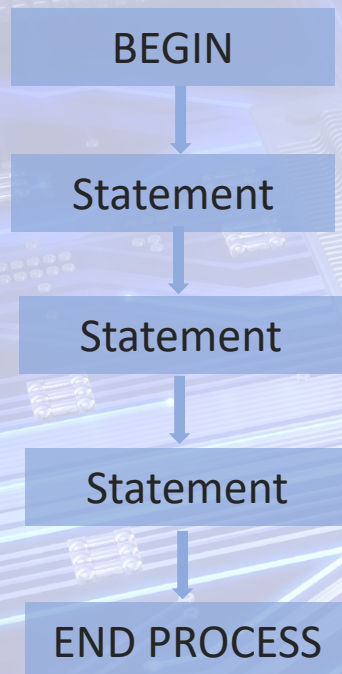
1. Introducción

- ❖ Los statements concurrentes de asignación de señales, sólo permiten describir el funcionamiento paralelo de los sistemas digitales, tales como los sistemas combinacionales
- ❖ Sin embargo, en la síntesis o modelado de los sistemas digitales, también deben describirse sistemas secuenciales:
 - Se describen de forma algorítmica
 - ✓ Su funcionamiento se especifica como una secuencia concreta de eventos de las señales de entrada
 - ✓ Ejemplos: una máquina de estados, un contador o un registro
- ❖ En VHDL se especifican los sistemas secuenciales mediante un process o un subprograma que contiene varios statements secuenciales y de asignación de valores a señales y/o variables
- ❖ Los process también se pueden emplear para describir sistemas combinacionales

2. Statement Process (Proceso)

❖ Características generales

- Es el elemento descriptivo más importante de VHDL
- Se usa en el estilo descriptivo algorítmico
- Al ejecutarse un proceso la ejecución de los statements que contiene se realiza de forma secuencial:
 - ✓ Se ejecutan de arriba hacia abajo según el orden de escritura.
 - ✓ El orden de escritura de los statements afecta a los resultados de la simulación y de la síntesis.
- En conjunto se considera un statement concurrente:
 - ✓ Por tanto, se puede usar tantos procesos y otros statements concurrentes, como sean necesarios para describir un diseño.
- Un proceso tiene dos estados:
 - ✓ Espera o suspensión
 - ✓ Ejecución



2. Statement Process (Proceso)

Sintaxis

```
[etiqueta:] process [(lista_de_sensibilidad)]  
    {Declaración_local}  
  
    Begin  
        { Statement_secuencial }  
        { Statement_secuencial }  
  
    End process [etiqueta];
```

■ Etiqueta

- ✓ Es opcional.
- ✓ Es un identificador que se usa para asignar un nombre al proceso.
- ✓ Debe terminar en dos puntos.

✓ Utilidad:

- ☐ Identificar el process durante la simulación.
- ☐ Distinguir variables locales con el mismo nombre que se usan en varios process.



2. Statement Process (Proceso)

```
[etiqueta:] process [(lista_de_sensibilidad)]  
    {Declaración_local}  
Begin  
    { Statement_secuencial }  
    { Statement_secuencial }  
End process [etiqueta];
```

- **Declaración local**

- Es opcional.
- Es la zona comprendida entre la lista de sensibilidad y la palabra clave **Begin**.
- Se declaran objetos, tipos de datos y subprogramas locales que sólo se pueden usar en el process.

- **Lista de sensibilidad**

- Es opcional.
- Es una lista de señales encerrada entre paréntesis.
 - ✓ Las señales se separan por comas.
- Especifica la condición de ejecución del proceso
 - ✓ Se ejecuta si se produce un evento en alguna de las señales incluidas en la lista de sensibilidad.
- Si no hay lista de sensibilidad el process se ejecuta continuamente.
 - ✓ Debe incluirse al menos un statement **wait** para suspender la ejecución del proceso.

2. Statement Process (Proceso)

```
[etiqueta:] process [(lista_de_sensibilidad)]  
    {Declaración_local}  
Begin  
    { Statement_secuencial }  
    { Statement_secuencial }  
End process [etiqueta];
```

• Cuerpo del proceso

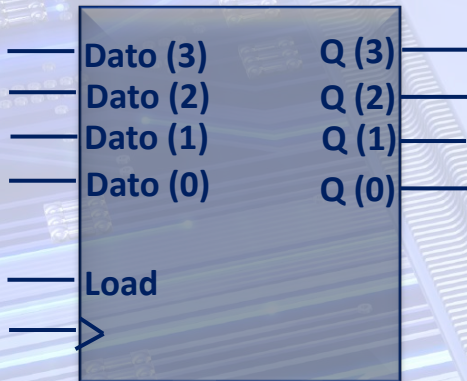
- Es obligatorio.
- Es la zona comprendida entre las palabras claves **Begin** y **End process**.
- Consta de statements secuenciales y de asignación de valores a señales y/o a variables.



2. Statement Process (Proceso) Funcionamiento

❖ Funcionamiento de un process

- Describe una serie de statements secuenciales y de asignación de valores a señales y/o variables, cuya ejecución depende de uno o más eventos.
- Ejemplo.** Proceso que describe un registro con carga paralela activo por flanco de subida:
 - ✓ No cambia de estado (carga) hasta que no se produzca un **flanco de subida en la señal de reloj** y esté activa la entrada **Load**.
 - ✓ Se debe producir un **evento en la entrada de reloj** para que se evalúe el funcionamiento.
 - ✓ **Descripción algorítmica:**
Si $\text{Clk} = \uparrow$ entonces
 Si $\text{load} = 1$ entonces
 $Q = D$
 De lo contrario $Q = Q$
 - ✓ El orden de ejecución está definido.
 - Se ejecutan los statements secuenciales **de arriba abajo**.



2. Statement Process (Proceso) Funcionamiento

- **Descripción algorítmica:**

Si $\text{Clk} = \uparrow$ entonces

Si $\text{load} = 1$ entonces

$Q = D$

De lo contrario $Q = Q$

- El tiempo de ejecución es 0. Esto implica:

- Se considera en conjunto como un statement concurrente.
- Si se usan varios process, todos se ejecutan en paralelo, y el tiempo de ejecución es el mismo indiferentemente de su tamaño.

Descripción del registro con carga paralela

process (Clk)

Begin

if Clk = '1' **and** Clk'event **then**

if load = '1' **then**

$Q \leq D;$

end if;

end if;

end process;

- El proceso se ejecuta si se
- produce un evento en Clk
- Detección del flanco de subida
- carga paralela

2. Statement Process (Proceso)

Activación de un process

❖ Process con lista de sensibilidad

- Un process con lista de sensibilidad se ejecuta siempre que se produzca un evento en cualquiera de las señales de la lista de sensibilidad.
- **Empleo de process con lista de sensibilidad**
 - ✓ Descripción de circuitos que responden a estímulos externos.
 - Circuitos combinacionales.
 - Circuitos secuenciales.
 - Mezcla de ambos.

▪ Ejemplo. Flip-flop D con clear asíncrono

```
process(Clear, Clk)
begin
    if Clear = '1' then
        Q <= '0';
    elsif Clk = '1' and Clk'event then
        Q <= D;
    end if;
end process;
```



2. Statement Process (Proceso)

Activación de un process

Flip-flop D con clear asíncrono

```
process(Clear, Clk)
begin
    if Clear = '1' then
        Q <= '0';
    elsif Clk = '1' and Clk'event then
        Q <= D;
    end if;
end process;
```

- Al producirse un evento en **Clear** o **Clk** se ejecuta el process.
- Se ejecutan los statements secuenciales de arriba (**begin**) a abajo (**end process**).
- La condición de **CLEAR** es prioritaria.
 - Si la entrada **Clear** es '1' la salida **Q** se pone a '0'.
 - Si **Clear** no es '1' se comprueba la entrada **Clk** para determinar si se ha producido un flanco de subida.
 - **Condición:**
 - ✓ Se produjo un evento
 - ✓ Valor actual = '1'
 - ✓ La comprobación del valor y de la condición de evento es la forma de detectar transiciones o flancos de reloj, tanto en la síntesis como en la simulación.

2. Statement Process (Proceso)

Activación de un process

Flip-flop D con clear asíncrono

```
process(Clear, Clk)
begin
    if Clear = '1' then
        Q <= '0';
    elsif Clk = '1' and Clk'event then
        Q <= D;
    end if;
end process;
```

- Una vez analizados los statements (end process) el process se suspende.
- Desde el punto de vista de la simulación el tiempo de ejecución de un process es 0.
- **En conjunto un process se considera un statement concurrente.**
- **Las señales se actualizan al suspenderse el process.**
- Se asigna el valor a las señales tras un retardo delta.
- **Las asignaciones a las variables dentro de un process se realizan inmediatamente.**

2. Statement Process (Proceso)

Activación de un process

```
process
begin

.....
F <= '1';
.....
wait until Clk = '1';
.....
.....
end process;
```

❖ Process sin lista de sensibilidad

- Se ejecuta siempre, por lo que debe incluirse un statement **wait** para que se actualicen los valores de las señales.
- Se comienza por el primer statement secuencial hasta el primer statement **wait**.
 - ✓ El process se suspende hasta que se cumpla la condición del statement **wait**.
 - ✓ Se actualiza el valor de la señal F.
 - F se pone a 1,
 - ✓ El process se reactiva y, si hay un solo **wait**, continúa la ejecución hasta **end process**.
 - ✓ Vuelve a ejecutarse de nuevo.

2. Statement Process (Proceso)

Activación de un process

❖ Process sin lista de sensibilidad

▪ Ejemplo

```
process  
begin  
    wait until Clk = '1';  
        dato_sin <= dato_asin;  
    wait until Clk = '1';  
        dato_val <= dato_sin;  
end process;
```

- La ejecución se suspende en **dos puntos**.
- El process no se suspende en el statement end process, sino que comienza a ejecutarse de nuevo.
- Al suspenderse el process en el statement wait, se actualiza el valor de las señales.
- Un evento en alguna de estas señales activará otro process que la emplee.

• Utilidad

- Describir circuitos o sistemas con múltiples señales de reloj.
- No están contemplados por la mayoría de las herramientas de síntesis actuales.
- Describir los estímulos de los test bench.



3. Statements secuenciales

❖ Statements secuenciales

- Los statements secuenciales **se emplean dentro de un process o un subprograma** (function o procedure).
- Se emplean para controlar el flujo de ejecución de los statements y la ejecución de los statements de asignación a señal, y/o variable.
- VHDL permite los siguientes statements secuenciales:
 - ✓ **IF-THEN-ELSE**
 - ✓ **CASE**
 - ✓ **LOOP**
 - ✓ **WAIT**



3. Statements secuenciales.

Statement IF-THEN-ELSE

❖ Statement IF-THEN-ELSE

- Es el statement de control más empleado en VHDL.
- **Utilidad**
 - ✓ Elegir una o ninguna de las secuencias de statements entre un conjunto dado de secuencias.
- **Sintaxis**

```
[etiqueta_if : ]  
  if primera_condición then  
    { secuencia1_statements }  
  { elsif segunda_condición then  
    { secuencia2_statements } }  
  [ else  
    { secuencia_defecto_statements } ]  
  end if [etiqueta_if];
```



3. Statements secuenciales.

Statement IF-THEN-ELSE

```
[etiqueta_if : ]  
  if primera_condición then  
    { secuencia1_statements }  
    { elsif segunda_condición then  
      { secuencia2_statements } }  
    [ else  
      { secuencia_defecto_statements } ]  
  end if [etiqueta_if];
```

❖ Sintaxis

- Un statement **if** comienza con la palabra clave **if** y termina con las palabras claves **end if**.
- Hay dos cláusulas opcionales: **elsif** y **else**.
- Las condiciones, que se especifican, se evalúan para un valor **booleano**:
 - ✓ Si una condición se evalúa a un valor **true**, se ejecuta la secuencia de statements correspondiente.
 - ✓ Si ninguna condición toma el valor **true**, entonces se ejecuta la secuencia de statements de la cláusula **else**.



3. Statements secuenciales.

Statement IF-THEN-ELSE

```
[etiqueta_if : ]  
  if primera_condición then  
    { secuencia1_statements }  
  { elsif segunda_condición then  
    { secuencia2_statements } }  
  [ else  
    { secuencia_defecto_statements } ]  
end if [etiqueta_if];
```

❖ Ejemplo1. Un solo statement if-then

```
if x < 10 then  
  c <= b;  
end if;
```

❖ Se evalúa la condición de la cláusula if (x < 10)

- Si x < 10 es **true se ejecuta** el statement que sigue a **then**:
 - ✓ Se asigna el valor de b a c
- Si x < 10 es **false (x >= 10) se salta** el statement de asignación y se ejecuta el que hay a continuación de **end if**.
 - ✓ c mantiene su valor.

3. Statements secuenciales.

Statement IF-THEN-ELSE

❖ Ejemplo2. Statement if incorrecto



```
process (S, B)
  signal O: std_logic;
begin
  if S then           --Incorrecto por no ser S del tipo boolean
    O <= B;
  end if;
end process;
```

❖ Statement if corregido



```
process (S, B)
  signal O: std_logic;
begin
  if S = '1' then     --El valor de la condición = es del tipo boolean
    O <= B;
  end if;
end process;
```


3. Statements secuenciales.

Statement IF-THEN-ELSE

Statement IF-THEN con cláusula ELSE

```
If condición then  
    Secuencia1_statements;  
else  
    Secuencia2_statements;  
end if;
```

- **Se evalúa la condición:**
 - ✓ Si es **true** se ejecuta la secuencia 1 de statements.
 - ✓ Si es **false** se ejecuta la secuencia 2 de statements.
- **Utilidad principal de la cláusula else**
 - ✓ Realizar la asignación por defecto a las señales que se usen en el process



3. Statements secuenciales.

Statement IF-THEN-ELSE

❖ Statement IF-THEN con cláusula ELSE

▪ Ejemplo. Mux de 2 a 1

```
process (S, E0, E1)
begin
    if S = '0' then
        Y <= E0;
    else
        Y <= E1;
    end if;
end process;
```

✓ Funcionamiento:

- Si S es 0 se asigna el valor de E0 a la señal Y.
- Si S es distinto de 0 se asigna el valor de E1 a Y.



3. Statements secuenciales. Statement IF-THEN-ELSE

```
process (S, E0, E1)
begin
  if S = '0' then
    Y <= E0;
  else
    Y <= E1;
  end if;
end process;
```

❖ Ejemplo. Mux de 2 a 1

▪ Otra descripción alternativa sin cláusula ELSE

```
process (S, E0, E1)
begin
  Y <= E1;
  if S = '0' then
    Y <= E0;
  end if;
end process;
```

❖ Conclusión

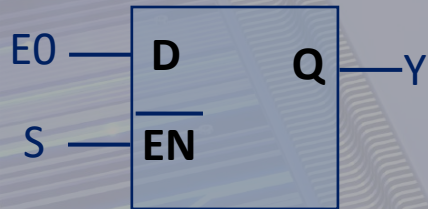
- Hay dos formas de especificar la secuencia de statements por defecto:
 - ✓ Mediante la cláusula else
 - ✓ Introducirlos antes del statement IF-THEN

❖ Funcionamiento

- Los statements se ejecutan según el orden de escritura que tienen en el process.
- No se asigna el valor a las señales hasta que no se suspenda el process:
 - ✓ El valor final de la señal está determinado por la última asignación.
- Por tanto, en ambos process se asigna **E1 a Y si S /= 0**

3. Statements secuenciales. Statement IF-THEN-ELSE

Componente sintetizado



❖ Statement if-then sin secuencia de statements por defecto

```
Process (S, E0)
begin
  if S = '0' then
    Y <= E0;
  end if;
end process;
```

■ Funcionamiento

- ✓ No es equivalente a las dos anteriores
 - Si $S = 0$, se asigna el valor de $E0$ a Y .
 - Si $S \neq 0$, Y mantiene su valor.
- ✓ Se define un latch

3. Statements secuenciales. Statement IF-THEN-ELSE

❖ Statement IF-THEN-ELSIF-ELSE

Elsif permite evaluar y **priorizar varias condiciones** para ejecutar **una única secuencia de statements**.

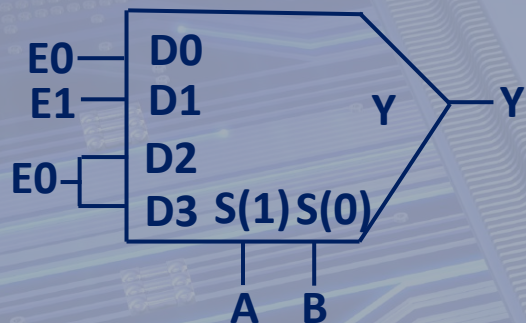
```
if condición1 then
    secuencia1;           -- Se ejecuta si condición1 es true
elsif condición2 then
    secuencia2;           -- Se ejecuta si condición1 es false y condición2 true
elsif condición3 then
    secuencia3;           -- Se ejecuta si condición1 y condición2 son false y
                           -- condición3 true
else
    secuencia4;           -- Se ejecuta si las tres condiciones son false
end if;
```

❖ Conclusión

- Al evaluarse las condiciones de arriba abajo se les asigna una prioridad a cada una.
- Condición1 es más prioritaria que condición2, y condición2 que condición3.

3. Statements secuenciales. Statement IF-THEN-ELSE

Componente sintetizado



❖ Statement IF-THEN-ELSIF-ELSE

▪ Ejemplo

```
if A = '1' then
    Y <= E0;
elsif B = '1' then
    Y <= E1;
else
    Y <= E0;
end if;
```

- Por asignar una prioridad a cada condición es semejante al statement when-else.
- Esta descripción también se puede sintetizar mediante un statement when-else.

3. Statements secuenciales. Statement IF-THEN-ELSE

❖ Statement IF-THEN anidados

- Un statement if-then puede contener cualquier statement secuencial de VHDL, incluso otros statements if-then.
- Se puede anidar múltiples niveles de if-then.

❖ Funcionamiento

- La secuencia de statements1 se ejecuta si son true condición_externa1 y condición_interna1.
- La secuencia de statements2 se ejecuta si es false condición_externa1 y true condición_interna2.

❖ Observación

- Se pueden sustituir por **elsif**

```
if condicion_externa1 then
  if condición_interna1 then
    statements1;
  end if;
else
  if condicion_interna2 then
    statements2;
  end if;
end if;
```

- ❖ Cada statement **IF** debe terminarse con su correspondiente cláusula **END IF**.

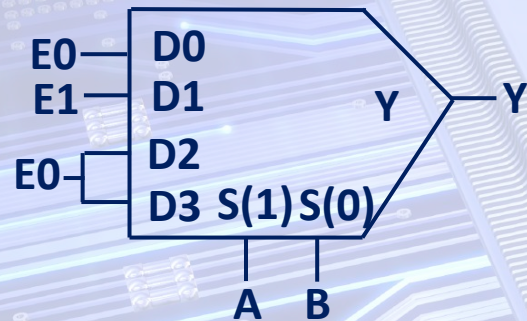


3. Statements secuenciales. Statement IF-THEN-ELSE

❖ Statement IF-THEN anidados

▪ Ejemplo. Mux de 4 a 1 con habilitación

```
process (E, En, S)
begin
  if En = '1' then
    if S = "00" then
      Y <= E0;
    elsif S = "01" then
      Y <= E1;
    elsif S = "10" then
      Y <= E2;
    else
      Y <= E3;
    end if;
  else
    Y <= '0';
  end if;
End process;
```



▪ La condición de habilitación debe cumplirse para evaluar los valores de las entradas de selección.

▪ Por eso, se describe como la condición externa, y se anida un statement if-then-elsif-else a ésta.

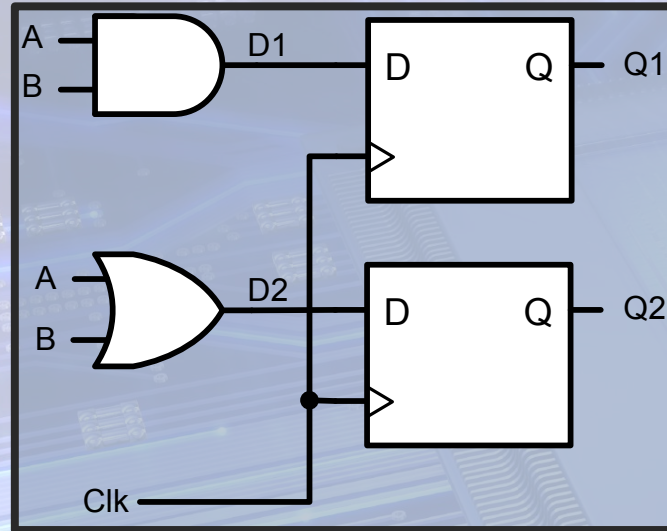
▪ Describe la asignación de la salida si enable está desactiva

4. Asignación de signals y variables

❖ Comparativa señal - variable

- La temporización de ambas difieren en un process.
- Una **variable** solamente se puede **usar dentro de un process o subprograma** en el que se declara, y una **señal** es visible en toda la arquitectura.
- Una variable es un elemento de almacenamiento local en contraposición a una signal que representa una interconexión entre circuitos o statements concurrentes, o un elemento de almacenamiento global.
- Ejecución de statements de asignación de valores a señales y variables en un process:
 - ✓ **Variables**
 - Se les asigna el valor inmediatamente.
 - ✓ **Señales**
 - Se genera una transacción y el valor de ésta se les asigna al suspenderse la ejecución del process (o subprograma).

4. Asignación de signals y variables. Casos de uso de señales y variables



❖ Casos de utilización de señal y variable

- Se modifica el valor de un objeto dentro de un process y éste debe emplearse posteriormente dentro del process antes de abandonarlo.
- Debe emplearse una variable.
- **Ejemplo. Descripción del circuito de la figura**
 - ✓ Si se describen las variables de excitación (D1 y D2) dentro del proceso se debe usar variables para D1 y D2.

```
Library ieee;
Use ieee.std_logic_1164.all;
Entity circuito is port (
  A, B, Clk : in std_logic;
  Q1, Q2 : out std_logic);
End;
```

```
architecture funcional of circuito
begin
  process (Clk)
    variable D1, D2 : std_logic;
  begin
    if Clk'event and Clk = '1' then
      D1 := A and B;
      D2 := A or B;
      Q1 <= D1;  -- Q1 <= A and B;
      Q2 <= D2;  -- Q2 <= A or B
    end if;
  end process;
End;
```



4. Asignación de signals y variables. Casos de uso de señales y variables

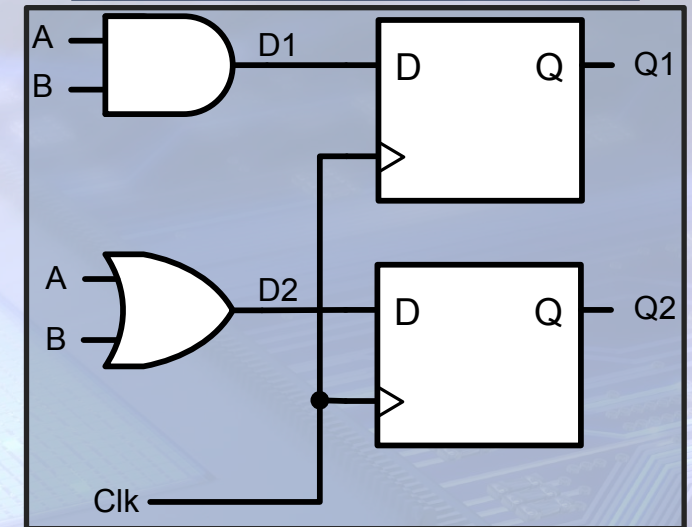
❖ Si se realiza la misma descripción con señales

```
architecture funcional of circuito
begin
  process (Clk)
    signal D1, D2 : std_logic;
  begin
    if Clk'event and Clk = '1' then
      D1 <= A and B;
      D2 <= A or B;
      Q1 <= D1;
      Q2 <= D2;
    end if;
  end process;
End;
```

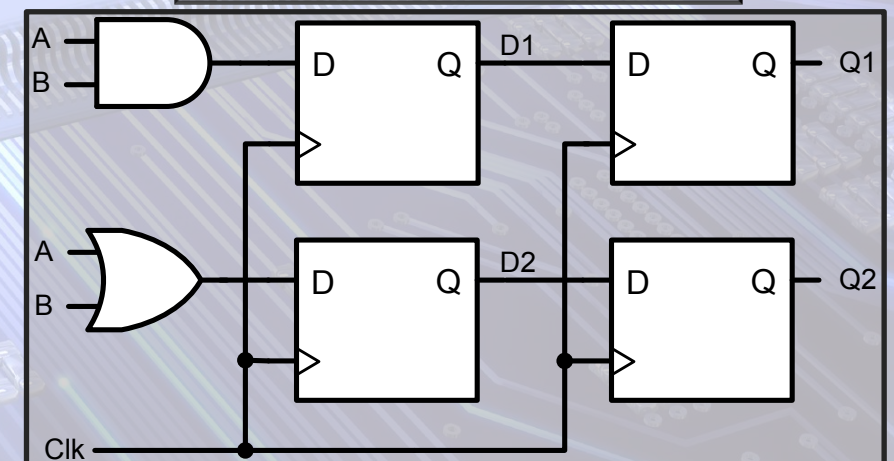
❖ **Conclusión**

- Se genera un flip-flop para D1 y D2

❖ Circuito a describir



❖ Circuito sintetizado



4. Asignación de signals y variables. Casos de uso de señales y variables

```
architecture funcional of circuito
begin
  process (Clk)
    signal D1, D2 : std_logic;
  begin
    if Clk'event and Clk = '1' then
      D1 <= A and B;
      D2 <= A or B;
      Q1 <= D1; -- Q1 = A and B(n-1)
      Q2 <= D2; -- Q2 = A or B(n-1)

    end if;
  end process;
End;
```

❖ Conclusión

- Se sintetizan cuatro flip-flops en total en vez de dos.

❖ Descripción de la ejecución del process

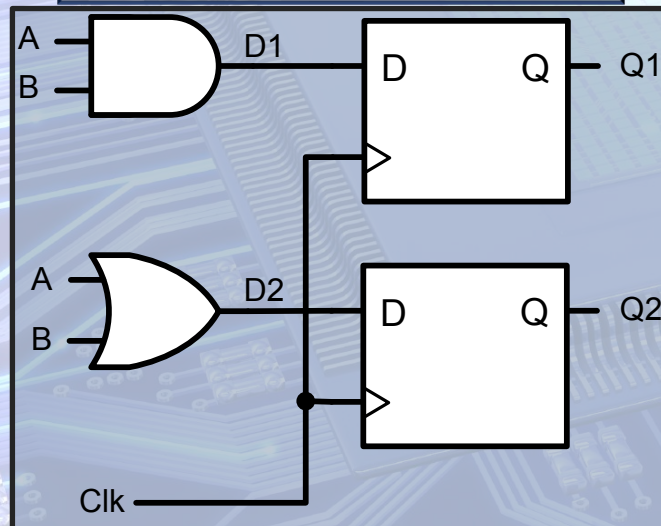
- ❖ Se genera una transacción para D1, D2, Q1 y Q2, cada vez que se produce un flanco de subida en Clk.
- ❖ **El valor de todas las señales se actualiza al suspenderse el process (end process).**
 - D1 se actualiza con la and lógica del valor que tienen A y B “en el flanco de subida”.
 - D2 se actualiza con la or lógica del valor que tienen A y B “en el flanco de subida”.
 - Q1 se actualiza con el valor que tenía D1 al empezar a ejecutarse el process, es decir con el que se le asignó en la anterior ejecución del process ⇒ anterior flanco de subida de Clk.
 - ✓ **Se sintetiza un flip-flop D tanto para D1 como para Q1**
 - Igualmente ocurre para Q2 con D2.

4. Asignación de signals y variables. Casos de uso de señales y variables

❖ El circuito se puede describir **sin variables. Dos opciones:**

- Asignando la función de excitación directamente a los puertos Q1 y Q2.
- Generando mediante otro process o statements concurrentes las funciones de excitación.

❖ **Circuito a describir**



architecture funcional of circuito

begin

process (Clk)

begin

if Clk'event and Clk = '1' then

Q1 <= A and B;

Q2 <= A or B;

end if;

end process;

End;

architecture funcional of circuito

signal D1, D2 : std_logic;

begin

D1 <= A and B;

D2 <= A or B;

process (Clk)

begin

if Clk'event and Clk = '1' then

Q1 <= D1;

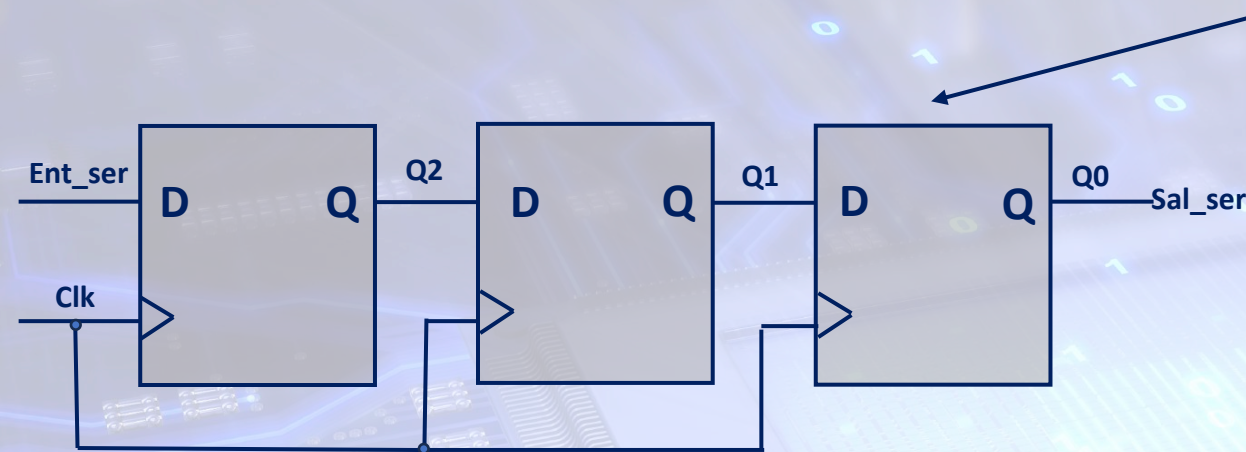
Q2 <= D2;

end if;

end process;

End;

4. Asignación de signals y variables. Casos de uso de señales y variables



```

Library ieee;
Use ieee.std_logic_1164.all;
Entity circuito is port (
    Ent_ser, Clk : in std_logic;
    Sal_ser : out std_logic
);
End;

```

❖ El statement de asignación de la salida debe ponerse fuera del **process**

❖ Casos de utilización de señal y variable

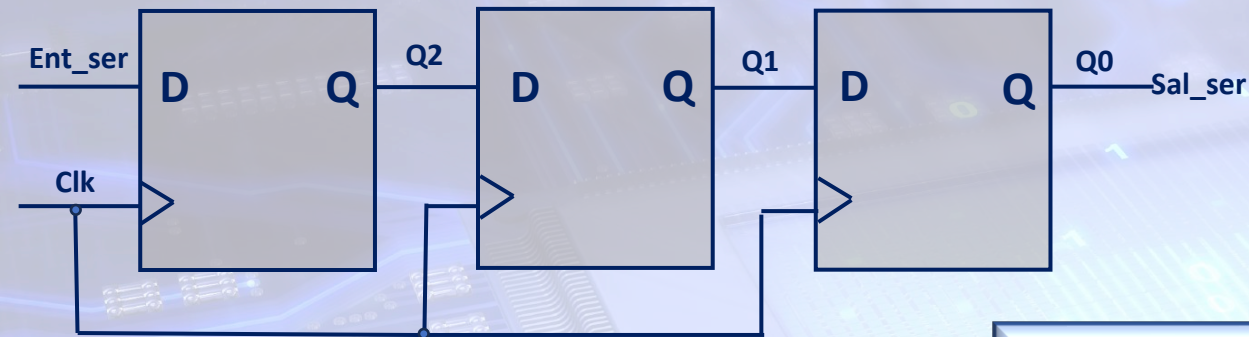
- Se declara una cadena de flip-flops mediante statements de asignación dentro de un process.
- Las salidas de los flip-flops se debe declarar como señales.
- Ejemplo. Descripción del circuito de la figura

```

architecture funcional of circuito
    signal Q2, Q1, Q0 : std_logic;
begin
    process (Clk)
    begin
        if Clk'event and Clk = '1' then
            Q2 <= Ent_ser;
            Q1 <= Q2;
            Q0 <= Q1;
        end if;
    end process;
    Sal_ser <= Q0 after 10 ns;
End;

```


4. Asignación de signals y variables. Casos de uso de señales y variables



❖ Casos de utilización de señal y variable

- Si se declaran como variables no se generan los flip-flops para Q2 y Q1.

- Al suspenderse el process se asigna el valor de **Ent_ser** a **Sal_ser**
- **Sal_ser** toma el valor de **Ent_ser** con cada flanco de subida de la señal de reloj
- El statement de asignación de la salida debe incluirse en el **process**

architecture funcional of circuito
begin

process (Clk)

variable Q2, Q1, Q0 : std_logic;

begin

if Clk'event and Clk = '1' then

Q2 := Ent_ser;

Q1 := Q2;

Q0 := Q1;

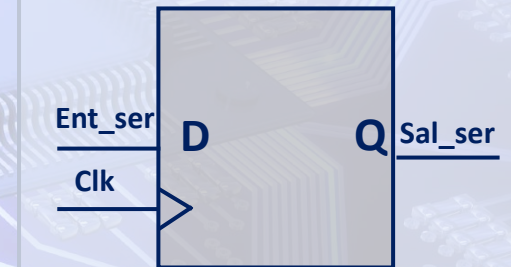
end if;

Sal_ser <= Q0 after 10 ns;

end process;

End;

Circuito sintetizado



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Statement CASE-WHEN

- **Selecciona una acción** (secuencia de statements y/o asignación de valores a señales y/o variables) entre varias mediante el **valor de una única expresión**.
- Es semejante a la asignación seleccionada de señal.
- **Sintaxis**

```
[etiqueta_case : ]  
case expresión is  
    when selección => secuencia_statements;  
    {when selección => secuencia_statements; }  
    when others => secuencia_statements;  
end case [etiqueta_case];  
  
opción de selección  
( Valor_expresión | rango_discreto | others) { | ... }
```



5. Statements secuenciales (continuación). Statement CASE-WHEN

Sintaxis

[etiqueta_case :]

case expresión **is**

when selección => secuencia_statements;

 { **when** selección => secuencia_statements; }

when others => secuencia_statements;

end case [etiqueta_case];

opción de selección

(Valor_expresión | rango_discreto | others) { | ... }

- La expresión devuelve un valor que es igual a alguno de selección de las cláusulas WHEN o a OTHERS.

- Se usa OTHERS como último valor para englobar los que no se hayan indicado.

- Los valores de las distintas selecciones deben ser mutuamente excluyentes.

• Funcionamiento

- Si la expresión es igual a un valor de selección se ejecuta la secuencia de statements de la cláusula when correspondiente.
- Se ejecuta una única secuencia de statements.
- Posteriormente, se transfiere el control al siguiente statement de la cláusula end case.



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Utilidad

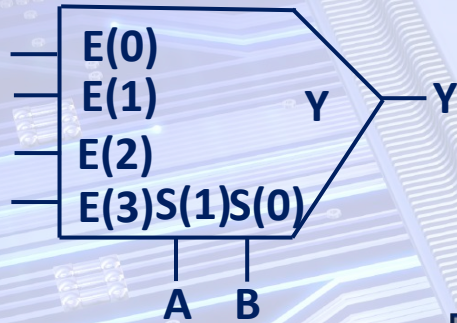
- Descripción de **máquinas de estado** y **circuitos combinacionales** a partir de su **tabla de verdad**
- De forma genérica se usa para describir cualquier **componente cuyo funcionamiento dependa de los distintos valores de una expresión**
 - ✓ En la síntesis se suele usar generalmente una **señal**
- **Ejemplos:**
 - ✓ Multiplexor
 - ✓ Registro universal tipo 74194
 - ✓ Decodificador BCD a 7 segmentos



3. Statements secuenciales (continuación). Statement WHEN-ELSE

Entidad

```
Library ieee;
Use ieee.std_logic_1164.all;
entity mux is port (
    E : std_logic_vector(3 downto 0);
    Sel : std_logic_vector(1 downto 0);
    Y : std_logic);
end mux;
```



Ejemplo. Mux de 4 a 1

Arquitectura

```
architecture funcional of mux is
begin
    process (E, S)
    begin
        case S is
            when "00" =>
                Y <= E(0);
            when "01" =>
                Y <= E(1);
            when "10" =>
                Y <= E(2);
            when others =>
                Y <= E(3);
        end case;
    end process;
end funcional;
```

- Se usa **OTHERS** como último valor para englobar los que no se hayan indicado:
 - "11" para la síntesis

5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Características

- Se deben indicar todos los valores de la señal de selección
 - ✓ Para los valores de selección que no se realice una asignación a las señales o variables, que se usen en el process, éstas mantienen su valor

- **Ejemplo**

architecture funcional of mux is

begin

process (E, sel)

begin

case sel is

when "00" =>

Y <= E(0);

when "01" =>

Y <= E(1);

when "10" =>

Y <= E(2);

end case;

end process;

end funcional;

- Al no incluirse **OTHERS**:
 - Si Sel = "11" el mux mantiene su valor de salida



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Características

▪ Repetición de una secuencia de statements para varios valores de selección:

- ✓ Se realiza la suma lógica de los valores de selección mediante el **operador |**
- ✓ **Others** se puede emplear para garantizar que se cubren todas las condiciones
 - Es la última selección
 - Especifica la secuencia de statements por defecto

▪ Ejemplo

architecture funcional of mux3a1 is

begin

 process (E, S)

 begin

 case S is

 when "00" =>

 Y <= E(0);

 when "01" | "10" =>

 Y <= E(1);

 when others =>

 Y <= E(2);

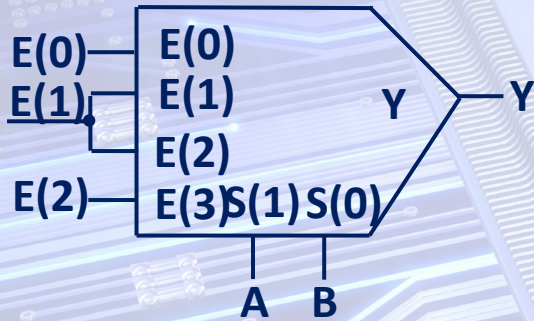
 end case;

 end process;

end funcional;

▪ Se asigna la entrada E(1) a la salida Y para los valores "01" y "10" de sel

▪ La cláusula **OTHERS** cubre el valor "11" para la síntesis



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Características

- No se puede formar un vector mediante el operador de concatenación en la lista de selección:
 - ✓ Debe ser una lista estática

■ Ejemplo. Descripción errónea

```
entity case_ej is port (  
    a,b: in std_logic_vector(2 downto 0);  
    Y : out std_logic_vector(2 downto 0));  
end case_ej;  
architecture funcional of case_ej is  
begin  
    process(a,b)  
    begin  
        case a & b is  
            --Error  
            when "000000" => Y <= "011";  
            when "001110" => Y <= "010";  
            when others => Y <= "000";  
        end case;  
    end process;  
end funcional;
```


5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Características

▪ Soluciones

- Declarar una señal y asignarle el resultado de la concatenación a&b

```
architecture funcional of case_ej is
    signal temp : std_logic_vector(5 downto 0);
begin
    temp <= a & b;
    process(temp)
    begin
        case temp is
            -- Correcto
            when "000000" => Y <= "011";
            when "001110" => Y <= "010";
            when others => Y <= "000";
        end case;
    end process;
end funcional;
```



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Características

■ Asignaciones múltiples

- ✓ En el **área concurrente** varias asignaciones a una misma señal genera una situación de **múltiples drivers**.
- ✓ En un **process** si se realizan varias asignaciones a una misma señal **no se genera una situación de múltiples drivers**.
 - Por ser la ejecución secuencial, **cada asignación anula la anterior**.
- Esto se puede emplear en un process para **asignar un valor por defecto a las señales**.

5. Statements secuenciales (continuación). Statement CASE-WHEN

▪ Ejemplo 1

architecture funcional **of** case_ej **is**

begin

process(a)

begin

case a **is**

when "00" =>

 Y1 <= '1'; Y2 <= '0'; Y3 <= '0';

when "10" =>

 Y1 <= '0'; Y2 <= '1'; Y3 <= '1';

when others =>

 Y1 <= '0'; Y2 <= '0'; Y3 <= '1';

end process;

end funcional;

- Es válido dentro de un solo process.
- Si una señal tiene asignaciones en varios process, se genera una situación de múltiples drivers.

❖ Características

- Estas dos descripciones son equivalentes, tanto en la simulación como en la síntesis:

▪ Ejemplo 1

architecture funcional **of** case_ej **is**

begin

process(a)

begin

 Y1 <= '0'; Y2 <= '0'; Y3 <= '0';

case a **is**

when "00" =>

 Y1 <= '1';

when "10" =>

 Y2 <= '1'; Y3 <= '1';

when others =>

 Y3 <= '1';

end process;

end funcional;

5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Cláusula null (nulo)

- **null** indica a la herramienta VHDL **no hagas nada**
- **Utilidad principal**
 - ✓ Se ha definido una asignación por defecto en un process y no se cambia para algún valor de selección de un statement case

- Para la selección others se mantiene la asignación por defecto
- Se puede eliminar null, pero de esta forma la descripción queda más clara

architecture funcional **of** case_nulo **is**
begin

process(a)

begin

Y1 <= '0'; Y2 <= '0'; Y3 <= '0';

case a **is**

when "00" =>

Y1 <= '1';

when "10" =>

Y2 <= '1'; Y3 <= '1';

when others => null;

end process;

end funcional;

5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Comparación case-when / if-then

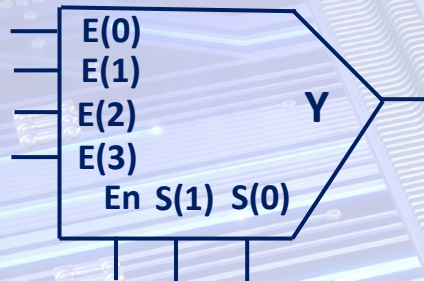
- El statement **case** se pueden usar, en algunos casos, como alternativa a **if-then**
- **Diferencia esencial entre case e if-then**
 - ✓ Los statements **if-then** implican una **prioridad de condiciones**
 - ✓ Un statement **case** no implica prioridad
- **Equivalencia con los statements concurrentes de asignación a señal:**
 - ✓ **case** = statements de asignación seleccionada
 - ✓ **if-then** = statements de asignación condicional
- **Utilidad principal en la síntesis**
 - ✓ Descripción de máquinas de estado



5. Statements secuenciales (continuación). Statement CASE-WHEN

❖ Anidamiento con otros statements secuenciales

- El statement **case** se pueden anidar con otros statements secuenciales, incluido el propio **if**
- **Ejemplo. Mux de 4 a 1 con entrada de habilitación**



architecture funcional of mux is
begin

process (E, En, S)

begin

Y <= '0';

if En = '1' then

case S is

when "00" => Y <= E(0);

when "01" => Y <= E(1);

when "10" => Y <= E(2);

when others => Y <= E(3);

end case;

end if;

end process;

end funcional;

5. Statements secuenciales (continuación). Statement WAIT

❖ Descripción

- Se emplea para **suspender la ejecución de un process o subprograma** (function o procedure)
- VHDL define tres condiciones en un statement **wait** para reiniciar la ejecución
 - ✓ **WAIT UNTIL** una expresión es **true**
 - ✓ **WAIT ON** cambio de señal
 - ✓ **WAIT FOR** un tiempo específico
- Se pueden combinar las tres condiciones



5. Statements secuenciales (continuación). Statement WAIT UNTIL

Sintaxis

Wait until expresión_lógica ;

❖ Funcionamiento

- Suspende la ejecución del **process** hasta que expresión_lógica devuelve un valor **true**.
- La expresión_lógica se evalúa si se produce un evento en alguna de sus señales.
- Crea una lista de sensibilidad de señales a partir de la expresión_lógica de una forma implícita.

❖ Ejemplo

Wait until $X * 10 > 100$;

- Si se produce un evento en X y su nuevo valor es mayor que 10 la ejecución continúa por el siguiente statement.



5. Statements secuenciales (continuación). Statement WAIT UNTIL

- **Aplicación**

- Describir test bench y lógica secuencial, tal como registros y flip-flops.
- **Ejemplo. Flip-flop D**

```
process  
begin  
    wait until clock = '1';  
    q <= d;  
end process;
```

- La ejecución se suspende hasta que clock cambie de valor y después de este cambio clock = '1'
- Por tanto, detecta un flanco de subida.
- Se genera una transacción con el valor de d a q.
- Se vuelve a ejecutar el proceso.
- Se vuelve a suspender el process y se actualiza el valor de q con d.
- Como el tiempo de ejecución es 0, q toma el valor de d en el flanco de subida de clock.

5. Statements secuenciales (continuación). Statement WAIT FOR

Sintaxis

Wait for expresión_tiempo;

❖ Funcionamiento

- **Suspende** la ejecución del process **durante el tiempo** indicado por expresión_tiempo.

❖ Ejemplo

wait for 10 ns;

-- suspende la ejecución durante 10 ns

wait for (a * (b + c));

-- suspende la ejecución el tiempo indicado

-- por la expresión

❖ Aplicación

- No se usa en la síntesis.
- **Se emplea en los test bench** para especificar los estímulos.



5. Statements secuenciales (continuación). Statement WAIT ON

Sintaxis

```
Wait on signal {, signal } ;
```

❖ Funcionamiento

- Espera a que se produzca un evento en alguna de las señales de la lista.
- Una vez detectado el evento la ejecución continúa por el statement siguiente a **wait on**.
- Genera una **lista de sensibilidad** para el process.
- No se suele usar en la síntesis.

5. Statements secuenciales (continuación). Statement WAIT ON

❖ Ejemplo. Flip-flop D con clear asíncrono

- Se comprueba si clear está activo.
 - ✓ Se genera una transacción con valor 0 a q.
- Si clear está desactivo se comprueba si se ha producido un flanco de subida en clock.
 - ✓ Se genera una transacción con el valor de d a q.
- Se suspende el process hasta que se produzca un evento en Reset y/o en clock.
 - ✓ Se actualiza el valor de q ('0' ó d).
- Se vuelve a ejecutar el process.

```
Process
Begin
  if clear = '1' then
    q <= '0';
  elsif clock = '1' and clock'event then
    q <= d;
  end if;
  wait on clear, clock;
End process;
```

- El statement wait on del final del proceso realiza la misma función que la lista de sensibilidad

- Process con lista de sensibilidad equivalente

```
Process (clear, clock)
Begin
  if clear = '1' then
    q <= '0';
  elsif clock = '1' and clock'event then
    q <= d;
  end if;
End process;
```


5. Statements secuenciales (continuación). Statement WAIT

- Resumen del statement **wait**
 - **Sintaxis**

```
[etiqueta:] wait [ on signal {, signal } ]  
[ until expresión_lógica ]  
[ for expresion_tiempo ] ;
```

- ✓ Se hace la or lógica de todas las condiciones definidas.
- **Síntesis**
 - ✓ Solamente se usa **wait on** y **wait until** por separado.
- **Simulación**
 - ✓ Es válida cualquier combinación

▪ Ejemplo

Wait until disparo = '1' **for** 1 ms;

- El tiempo de la cláusula **for** actúa como timeout.
 - ✓ Indica el **tiempo máximo** que estará suspendido el proceso.
- El proceso se suspende hasta que sea **true** una de las dos condiciones indicadas:
 - ✓ Flanco de subida en la señal disparo.
 - ✓ Ha transcurrido un milisegundo.

5. Statements secuenciales (continuación). Statement WAIT

❖ Conclusión

- **wait until** y **wait on** generan una lista de sensibilidad.
- Un process con lista de sensibilidad es equivalente a un process sin lista con un statement **wait on** en su parte final.
- En un process con lista de sensibilidad no se puede emplear statements **wait until** ni **wait on**.
- Un process sin lista de sensibilidad debe contener al menos un statement **wait** para suspender la ejecución, de forma que se actualicen las señales.
- El process con lista de sensibilidad es el formato más estandarizado para describir sistemas secuenciales sintetizables.



5. Statements secuenciales (continuación). Statement FOR LOOP

❖ Descripción

- Especifica un número concreto de iteraciones de una secuencia de statements.

▪ Sintaxis

[etiqueta_loop:] **for** identificador **in** rango discreto **loop**

secuencia_statements

end loop [etiqueta_loop];

▪ Ejemplo

Type array_int **is** array(1 **to** 10) **of** integer;

signal Tabla : array_int;

Begin

process

variable cuadrado : array_int;

Begin

Wait until disparo = '1';

For i **in** 1 **to** 10 **loop**

cuadrado(i) := i * i;

End loop;

Tabla <= cuadrado;

End process;

- ❖ El identificador hace referencia a una variable índice (i) y el rango discreto (1 **to** 10) el número de veces que se ejecuta el bucle:

- **Rango ascendente**, mediante la palabra clave **to**.
- **Rango descendente**, mediante la palabra clave **downto**.

- ❖ La variable índice (i) es **local** y **se declara automáticamente**

- ❖ **Se puede leer** la variable índice dentro del bucle **for**, **pero no se le puede asignar** ningún valor.

- Se comienza asignando a la variable índice el valor de la izquierda del rango, y posteriormente con cada iteración se le asigna los valores sucesivos según el rango definido.

5. Statements secuenciales (continuación). Statement FOR LOOP

• Otras características

▪ Bucles anidados

- ✓ La secuencia de statements de un bucle for puede contener statements de cualquier tipo.
- ✓ Se puede anidar los bucles for en múltiples niveles.

▪ Ejemplo

- Se emplean **dos bucles for anidados** en un statement if-then
- Si inicio se activa ('1'):
 - ✓ Se calcula la **tabla de multiplicar** de los números decimales.
 - ✓ Se almacena en la señal **tabla**.

architecture funcional of loop_ani is

type entero_array **is** array(0 to 9) **of** integer;

type entero_biarray **is** array(0 to 9) **of** entero_array;

signal tabla : entero_biarray;

begin

process(inicio)

variable temp : entero_biarray;

begin

if inicio = '1' **then**

for i in 0 to 9 **loop** -- bucle externo

for j in 0 to 9 **loop** -- bucle interno

temp(i)(j) := i * j;

end loop;

end loop;

end if ;

tabla <= temp;

end process;

end loop_ani;

5. Statements secuenciales (continuación). Statement FOR LOOP

❖ Otras características

- Puede existir **otro objeto con el mismo nombre** dentro del proceso.

- ✓ Se consideran **variables distintas**.

- ✓ Se acceden según el contexto:

- **Dentro del bucle for** la variable índice **oculta** el otro objeto.

- **Utilidad de la etiqueta del bucle**

- ✓ Distinguir entre la variable índice del bucle y otro objeto del mismo nombre.

- ✓ Si se emplean varios bucles, indicar el que debe terminarse.

▪ Ejemplo

process

variable i, b, c, d : integer;

Begin

i := 10;

For i in 0 to 7 loop

b := i;

End loop;

c := i; -- c = 10

d := b; -- d = 7

End process;

5. Statements secuenciales (continuación). Statement FOR LOOP

❖ Utilidad

▪ Síntesis

- ✓ Descripción de circuitos cuyo comportamiento se puede describir de forma iterativa, como decodificadores binarios, sumadores, etc.

▪ Simulación

- ✓ Modelación de componentes iterativos.
- ✓ Creación de test bench.

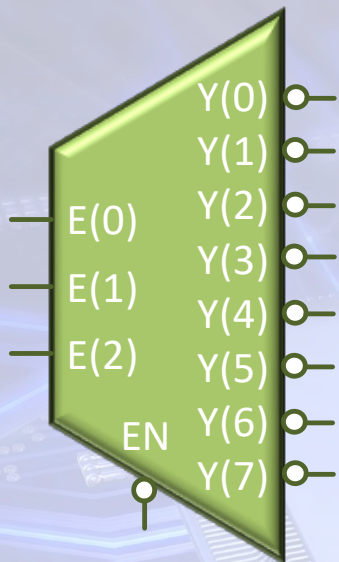
❖ Ejemplo test bench

- Genera los estímulos para simular un circuito combinacional de 4 variables de entrada

```
process
begin
  for i in 0 to 15 loop
    Entrada <= std_logic_vector(to_unsigned(i,4));
    wait for 100 ns;
  end loop;
end process;
```



5. Statements secuenciales (continuación). Statement FOR LOOP



- **Utilidad**

- **Ejemplo de descripción sintetizable**

- ✓ Decodificador binario de 3 a 8 con entrada de habilitación

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity Decod is port (
    En: in std_logic ;
    E: in std_logic_vector ( 2 downto 0 );
    Y: out std_logic_vector ( 7 downto 0 ) );
end;
```

architecture Funcional of Decod is

begin

process (En, E)

begin

Y <= (others => '1');

If En = '0' then

For i in 0 to 7 loop

if i = Conv_integer(E) then

Y(i) <= '0';

end if;

End loop;

End if;

End process;

end Funcional;

5. Statements secuenciales (continuación). Statement FOR LOOP

- Ejemplo. Generador de paridad par de n bits

▪ Entidad

```
library ieee;  
use ieee.std_logic_1164.all;  
entity paridad_Nbits is  
    generic (N : natural := 4);  
    port (  
        D : in std_logic_vector(0 to N-1);  
        EVEN : out std_logic);  
end paridad_Nbits;
```

▪ Arquitectura

```
architecture funcional of paridad_Nbits is  
begin  
    process(D)  
        variable temp : std_logic;  
    begin  
        temp := '0';  
        for i in 0 to N-1 loop  
            if D(i) = '1' then  
                temp := not temp; --temp es una variable  
            end if;  
        end loop;  
        EVEN <= temp;  
    end process;  
end funcional;
```



5. Statements secuenciales (continuación). Statement LOOP. EXIT

❖ Statement EXIT

- Se usa para terminar un bucle antes de que se cumpla la condición.
 - ✓ Ha ocurrido un error significativo en la ejecución.
 - ✓ Ha terminado todo el procesamiento.
- **Sintaxis**

```
Exit [etiqueta_loop] [ when condición ];
```

▪ **Funcionamiento**

- ✓ Se evalúa la condición y si ésta es true se sale del bucle
- ✓ La condición es opcional:
 - ☐ Si no se incluye se sale del bucle al ejecutarse el statement exit.



5. Statements secuenciales (continuación). Statement FOR LOOP. EXIT

❖ Ejemplo

- Realiza una operación aritmética concreta en la que se decrementa sucesivamente **a**.
 - Como **a** es un port debe utilizarse una **variable** en el process, **entero_a**.
- Se termina el cálculo cuando **entero_a** sea menor o igual que 0, o se realice 10 veces.

```
entity loop_real is port (a : integer);
end loop_real;

architecture funcional of loop_real is
    type array_real is array(0 to 9) of real;
    signal Y : array_real;
begin
    process (a)
        variable q : array_real;
        variable entero_a : integer;
    begin
        entero_a := a;
        for i in 1 to 10 loop
            exit when entero_a <= 0; --menor o igual que 0
            entero_a := entero_a -1 ;
            q(i) := 3.1416 / real(entero_a * i);
        end loop;
        y <= q;
    end process;
end funcional;
```


5. Statements secuenciales (continuación). Statement FOR LOOP. EXIT

- **Funcionamiento con bucles anidados**
 - Si se ejecuta un statement **exit** de un statement loop anidado (interno) con otro statement loop, **solamente se sale del statement loop más interno.**
 - Para terminar otro bucle, que no es el más interno, **debe especificarse la etiqueta del bucle** en el statement exit.

```
process (a)
  variable q : array_real;
  variable entero_a : integer;
begin
  primer_bucle : for i in 0 to 100 loop
    segundo_bucle : for j in 1 to 10 loop
      .....
      exit ;           --sale del segundo bucle solamente
      .....
      exit primer_bucle; --sale de ambos bucles
    end loop;
  end loop;
end process;
```



5. Statements secuenciales (continuación). Statement WHILE LOOP

- **Descripción**

- Se especifica la condición de repetición.
- **Sintaxis**

```
[etiqueta_loop:] while condición loop  
    secuencia_statements  
End loop [etiqueta_loop];
```

- ❖ **Funcionamiento**

- La condición se comprueba antes de cada iteración del bucle, incluyendo la primera.
 - ✓ Si la condición es **true** se repite.
 - ✓ Si la condición es **false** se termina el bucle.



5. Statements secuenciales (continuación). Statement WHILE LOOP

❖ Utilidad en la síntesis

- Solamente está permitido por las herramientas de simulación, y se emplea en la **generación de test bench**.
- Las herramientas de síntesis no suelen permitir este tipo de statement, porque no tiene un equivalente directo en hardware.
- **Ejemplo. Test bench**

- Describe una señal de reloj para la simulación de un diseño mediante un **test bench**
- El bucle se repite siempre que no se haya llegado al **final del test** (**fin** /= '1') o no se haya producido un error (**flag_error** /= '1').
- **Flag_error** y **fin** son señales generadas por otro process del test bench o por el UUT.

```
process
begin
    while flag_error /= '1' and fin /= '1' loop
        Clk <= not Clk;
        wait for Periodo_Clk/2;
    end loop;
end process;
```


5. Statements secuenciales (continuación). Statement LOOP

❖ Descripción

- Es un statement loop que no incluye un esquema de repetición, por lo que **se repite indefinidamente**.
- Se incluye una **condición de salida (Exit)**.
- **Utilidad en la síntesis**
 - ✓ Un bucle infinito no tiene un equivalente directo en hardware, por lo que no es directamente sintetizable.
- **Ejemplo. Otra descripción del test bench anterior**

```
process
begin
  loop
    Clk <= not Clk;
    wait for Periodo_Clk/2;
    exit when flag_error = '1' or fin = '1'
  end loop;
end process;
```



6. Procesos concurrentes

❖ Los process contienen statements secuenciales pero **en conjunto son statements concurrentes** dentro de una architecture.

- Se pueden emplear **varios procesos** para describir sistemas concurrentes.
- **No puede realizarse asignaciones a una misma señal en dos procesos.**

▪ Ejemplo

- Se describe una función and registrada de forma diferente en dos process

▪ Entidad

```
library ieee;
use ieee.std_logic_1164.all;
entity and_reg is port
    (a, b, clk, reset : std_logic;
     q, qbis : out std_logic );
end and_reg;
```

▪ Arquitectura

```
architecture funcional of and_reg is
begin
```

```
P1: process (clk, reset)
```

```
begin
```

```
    if reset = '1' then
```

```
        q <= '0';
```

```
    elsif clk = '1' and clk'event then
```

```
        q <= a and b;
```

```
    end if;
```

```
end process;
```

```
P2: process (clk, reset)
```

```
    variable salida : std_logic;
```

```
begin
```

```
    if reset = '1' then
```

```
        salida := '0';
```

```
    elsif clk = '1' and clk'event then
```

```
        salida := a nand b;
```

```
        salida := not salida;
```

```
    end if;
```

```
    qbis <= salida;
```

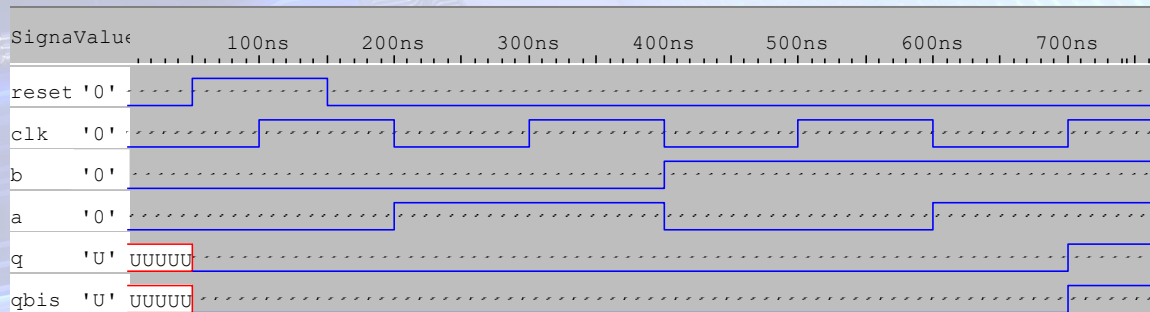
```
end process;
```

```
end ;
```


6. Procesos concurrentes

- Ambos process se activan al mismo tiempo cuando se produzca un evento en clk y/o reset.
- El tiempo de ejecución de ambos es el mismo.

Simulación



- El comportamiento de ambos process es el mismo, ya que ambas salidas tienen siempre el mismo valor.
- Ambas salidas cambian al mismo tiempo.
 - ✓ Los dos process se ejecutan al mismo tiempo.

Arquitectura

architecture funcional of and_reg is
begin

P1: process (clk, reset)

begin

if reset = '1' then

q <= '0';

elsif clk = '1' and clk'event then

q <= a and b;

end if;

end process;

P2: process (clk, reset)

variable salida : std_logic;

begin

if reset = '1' then

salida := '0';

elsif clk = '1' and clk'event then

salida := a nand b;

salida := not salida;

end if;

qbis <= salida;

end process;

end ;

6. Procesos concurrentes

Descripción de un test bench mediante dos process

- Hay tres statements concurrentes, los dos process y el de instanciación del componente contador.
 - ✓ El primer process genera la señal de reloj del contador.
 - ✓ El segundo process genera los estímulos correspondientes a las señales de reset y habilitación (EN).

architecture Estimulos **of** contador **is**

component conta4en **port**

(Clk, Reset, EN : **in** std_logic;

Cuenta : **out** std_logic_vector(3 **downto** 0));

end component;

constant Periodo_Clk : Time := 20 ns;

signal Clk, RESET, EN : std_logic;

signal Cuenta : std_logic_vector(3 **downto** 0);

begin

U0 : conta4en **port map** (Clk => Clk, Reset => Reset,
EN => EN, Cuenta => Cuenta);

process -- genera la señal de reloj
begin

Clk <= '1';

wait for periodo_Clk/2;

Clk <= '0';

wait for Periodo_Clk/2;

end process;

process -- genera el resto de estímulos
begin

Reset <= '0';

EN <= '1';

wait for Periodo_Clk/3;

Reset <= '1';

wait for Periodo_Clk;

Reset <= '0';

wait for Periodo_Clk*10;

EN <= '0';

wait for Periodo_Clk*3;

EN <= '1';

wait;

end process;

end Estimulos;

7. Metodología de uso de los PROCESS

- El process es el statement de VHDL más usado en la descripción de los componentes o sistemas digitales, tanto en la síntesis como en la simulación.
- Permite describir:
 - ✓ Sistemas combinacionales.
 - ✓ Sistemas secuenciales síncronos con inicialización asíncrona.
 - ✓ Sistemas secuenciales síncronos con inicialización síncrona.
- Metodología general:
 - ✓ Usar **process** con lista de sensibilidad.
 - ✓ Incluir en la lista de sensibilidad **solamente las señales que provoquen un cambio de valor** en las señales a las que se les asigne un valor en el process.

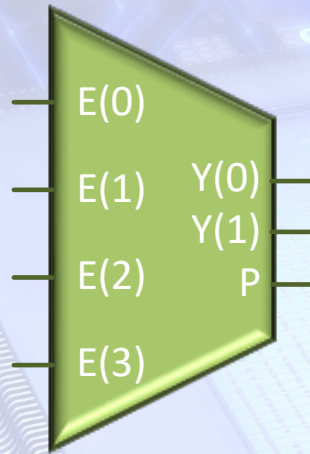
7. Metodología de uso de los PROCESS

- Los process permiten, en algunos casos, describir de una **forma más clara el comportamiento de los sistemas combinacionales** que los statements concurrentes de asignación de valores a señales.
 - ✓ La descripción del comportamiento se realiza de forma **algorítmica**.
- **Metodología:**
 - ✓ Incluir en la **lista de sensibilidad** del process **todas las señales que se lean** en el process, es decir, todas las entradas del sistema combinacional.
 - ✓ Los **statements de asignación a las salidas** deben cubrir todas las **posibles combinaciones de valores** de las entradas del process o entradas del sistema combinacional.
 - ✓ Si se emplean **variables** en el process hay que **asignarles un valor antes de leer** sus contenidos.

7. Metodología de uso de los PROCESS.

Descripción de sistemas combinacionales

Ejemplo. Codificador binario de 4 a 2 sin prioridad



```
library ieee;
use ieee.std_logic_1164.all;
entity cod_bin4a2 is port (
    E : std_logic_vector(3 downto 0);
    P : out std_logic;
    Y : out std_logic_vector(1 downto 0) );
end cod_bin4a2;
```

Architecture funcional of cod_bin4a2 is

Begin

Process (E)

Begin

P <= '0';

Y <= "00";

Case E is

when "0000" => P <= '1';

when "0001" =>

when "0010" => Y <= "01";

when "0100" => Y <= "10";

when "1000" => Y <= "11";

when others =>

end case;

end process;

End funcional;

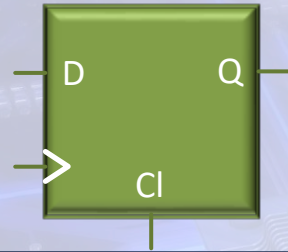
7. Metodología de uso de los PROCESS.

Descripción de sistemas secuenciales síncronos

- Es la aplicación principal de los process en la síntesis.
- Mediante process se suele describir **flip-flops, registros, contadores y máquinas de estado**.
- También se puede describir otros componentes secuenciales, como los **latches**.
- La descripción del comportamiento se realiza de forma **algorítmica**.
- Permite describir sistemas secuenciales con **inicialización**:
 - ✓ **Síncrona**
 - ✓ **Asíncrona**
- **Metodología**:
 - ✓ Escribir un process que **no incluya todas las entradas en la lista de sensibilidad**.
 - ✓ **Usar statements if-then-elsif incompletamente especificados**, de forma que una o más señales mantengan su valor bajo ciertas condiciones.
 - ✓ Usar una o más variables de forma que **mantengan un valor** entre las iteraciones del process.

7. Metodología de uso de los PROCESS.

Descripción de sistemas secuenciales síncronos



- El statement incompleto if-then-elsif especifica que la señal Q mantendrá su valor, si:
 - ✓ Clear está desactiva y no se produce un flanco de subida en la señal de reloj.

❖ Ejemplo

▪ Flip-flop con Clear asíncrono

- ✓ Al activarse el clear asíncrono se pone a cero el flip-flop indiferentemente de la señal de reloj.
 - La señal de clear debe incluirse en la lista de sensibilidad.
- ✓ Forma más común de especificar el reloj de un flip-flop:
 - Statement if-then-else
 - Condición: `clk'event and clk = '1'`

```
library ieee;
use ieee.std_logic_1164.all;
entity ffd_cl_asin is port (
    Clk, Clear, D : std_logic;
    Q : out std_logic );
end ffd_cl_asin;
```

Architecture funcional of ffd_cl_asin is Begin

```
    Process (Clear, Clk)
        If Clear = '1' then
            Q <= '0';
        elsif Clk'event and Clk = '1' then
            Q <= D;
        end if;
    end process;
End funcional;
```

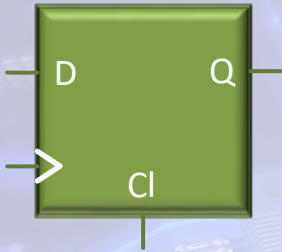

7. Metodología de uso de los PROCESS.

Descripción de sistemas secuenciales síncronos

❖ Ejemplo

▪ Flip-flop con Clear síncrono

- ✓ El flip-flop se pone a cero si clear está activa y se produce un flanco activo en la señal de reloj.
 - Por tanto, sólo se incluye la señal de reloj en la lista de sensibilidad del process.



```
library ieee;
use ieee.std_logic_1164.all;
entity ffd_cl_sin is port (
    Clk, Clear, D : std_logic;
    Q : out std_logic );
end ffd_cl_asin;
```

- El statement incompleto if-then externo especifica que la señal Q mantendrá su valor si no se produce un flanco de subida en la señal de reloj

Architecture funcional of ffd_cl_sin is Begin

Process (Clk)

```
    If Clk'event and Clk = '1' then
        If Clear = '1' then
            Q <= '0';
        else
            Q <= D;
        end if;
    end if;
end process;
End funcional;
```


8. Atributos

❖ Concepto de atributo

- Proporcionan **información adicional sobre un objeto** (señal o variable) o **un tipo de dato**

❖ Tipos de atributos

- **Los predefinidos en el estándar 1076**

- ✓ Se especifican varios tipos según el resultado que devuelven:
 - **Valor**
 - **Función**
 - **Señal**
- ✓ Características básicas:
 - **Se aplican a un prefijo**, que puede ser el nombre de una señal, una variable o un tipo o subtipo de dato.
 - Se relacionan con el prefijo mediante la **comilla simple (')**
 - **Wait until** `clk = '1' and clk'last value = '0'`;

- **Los definidos por el usuario**

8. Atributos. Atributos de tipo valor

- **Atributos de tipo valor**
 - Devuelven un valor explícito.
 - Se aplican a un **tipo o subtipo de dato**.
- **Atributos más comunes**
 - **'Left**
 - ✓ Devuelve el límite izquierdo de un tipo o subtipo.
 - **'Right**
 - ✓ Devuelve el límite derecho de un tipo o subtipo.
 - **'High**
 - ✓ Devuelve el límite superior de un tipo o subtipo.
 - **'Low**
 - ✓ Devuelve el límite inferior de un tipo o subtipo.
 - **'Length**
 - ✓ Devuelve longitud (número de elementos) de un array.<<

8. Atributos. Atributos de tipo valor

Ejemplos

```
type bit_array is array (0 to 5) of bit;  
type color is (azul, blanco, rojo, negro);  
variable L : integer := bit_array'left;           --L = 0  
variable color1 : color := color'left;           --color1 = azul  
variable R : integer := bit_array'right;          --R = 5  
variable color2 : color := color'right;          --color2 = negro  
variable H : integer := bit_array'high;           --H = 5  
variable color3 : color := color'high;           --color3 = negro  
variable Lw : integer := bit_array'low;           --Lw = 0  
variable color4 : color := color'low;            --color4 = azul  
variable LEN : integer := bit_array'length;       --LEN = 6
```


8. Atributos. Atributos de tipo valor

❖ Ejemplo de utilización de 'right y 'left.

- Permiten describir un registro de desplazamiento o rotación de n bits.
- Se usan los atributos para determinar los índices de la izquierda y derecha de un array.

```
--Registro de desplazamiento a la izquierda
architecture behavior of reg_desp is
begin
  reg: process (Clk)
  begin
    if Clk = '1' and Clk'event then
      Qreg <= Qreg(Qreg'left +1 to Qreg'right) & '0';
    end if;
  end process;
end behavior;
```


8. Atributos. Atributos de tipo valor

❖ Ejemplo de utilización de 'length y array no restringido.

- Se describe una puerta AND lógica de n entradas.

- El componente se usa en el nivel superior de un diseño jerárquico.
- Al instanciar el componente se le asigna a la entrada "e" un array restringido.
 - ✓ Se definen los índices del array.
- No se usa un generic.

- Si se usa un generic para definir el número de entradas no es necesario usar atributos.

```
entity and_n_entradas is
  port ( e : in std_logic_vector;
         y : out std_logic);
end and_n_entradas;
architecture behavioral of and_n_entradas is
  begin
    and_m : process (e) is
      variable result : std_logic;
    begin
      result := e(0);
      if ( e'length > 1) then
        for i in 1 to (e'length - 1) loop
          result := result and e(i);
        end loop;
      end if;
      y <= result;
    end process;
  end behavioral;
```


8. Atributos. Atributos de función array

❖ Atributos de función de array

- Devuelven información de los límites de un array.
- Si **A** es un array y **N** un entero entre 1 y el número de dimensiones de **A**, se distinguen los siguientes atributos:

❖ Atributos más comunes

- **A'Left(N)**
 - ✓ Devuelve el límite izquierdo del rango de índices de la dimensión **N** de **A**.
- **A'Right(N)**
 - ✓ Devuelve el límite derecho del rango de índices de la dimensión **N** de **A**.
- **A'High(N)**
 - ✓ Devuelve el límite superior del rango de índices de la dimensión **N** de **A**.
- **A'Low(N)**
 - ✓ Devuelve el límite inferior del rango de índices de la dimensión **N** de **A**.
- **A'Length(N)**
 - ✓ Devuelve la longitud del rango de índices de la dimensión **N** de **A**.
- **A'Range(N)**
 - ✓ Devuelve el rango de índices de la dimensión **N** de **A**.

8. Atributos. Atributos de función array

Ejemplos

```
type bit_array is array(1 to 4, 15 downto 0) of bit;  
variable L : integer := bit_array'left(1)           --L = 1  
variable R : integer := bit_array'right(2)          --R = 0  
variable H : integer := bit_array'high(2)           --H = 15  
variable Lw : integer := bit_array'low(1)           --Lw = 1  
variable Ra : integer := bit_array'range(1)         --Ra = 1 a 4  
variable Lg : integer := bit_array'length(1)        --Lg = 4
```


8. Atributos. Atributos de función array

- ❖ **Ejemplo de utilización de 'range y array no restringido.**
 - Se describe una puerta AND lógica de n entradas.

- El componente se usa en el nivel superior de un diseño jerárquico.
- Al instanciar el componente se le asigna a la entrada "e" un array restringido.
 - ✓ Se definen los índices del array.
- No se usa un generic.
- Si se usa un generic para definir el número de entradas no es necesario usar atributos.

```
entity and_n_entradas is  
  port ( e : in std_logic_vector;  
        y : out std_logic);  
end and_n_entradas;  
architecture behavioral of and_n_entradas is  
  begin  
    and_m : process (e) is  
      variable result : std_logic;  
      begin  
        result := '1';  
        for i in e'range loop  
          result := result and e(i);  
        end loop;  
        y <= result;  
      end process;  
  End behavioral;
```


8. Atributos. Atributos de función de señal

❖ Atributos de función de señal

- Devuelven información sobre el comportamiento de las señales.

❖ Atributos más comunes

- Sólo 'Event tiene utilidad en la síntesis

❖ 'Event

- Devuelve el valor **true** si se produjo un **evento** (cambio de valor) en la señal durante el ciclo actual de simulación

- De lo contrario devuelve un valor **false**

```
process (Clk)
begin
    if Clk = '1' and Clk'event then
        Q <= D;
    end if;
end process;
```

❖ 'Last_value

- Devuelve el valor de la señal antes del último evento

```
process
    variable V : std_logic;
begin
    Q <= '1';
    Wait for 10 ns;
    Q <= '0';
    Wait for 10 ns;
    V := Q'last_value;    --V = '1'
    ....
end process;
```


8. Atributos. Atributos de usuario

- **Atributos de usuario**

- Para usar un atributo de usuario, primero debe declararse y después especificarse.
- Sintaxis de declaración de un atributo

Attribute nombre_atributo : tipo_dato;

- Sintaxis de especificación de un atributo

Attribute nombre_atributo **of** nombre_elemento : clase **is** valor;

- Donde:
 - ✓ Tipo_dato es cualquier tipo de dato de VHDL: integer, std_logic, boolean, etc.
 - ✓ Clase del elemento, puede ser type, signal, function, etc.

8. Atributos. Atributos de usuario

❖ Utilidad

- Los suelen usar las herramientas de síntesis para controlar el proceso de síntesis de los ficheros fuente.

▪ Ejemplos:

- ✓ El atributo **syn_enum_encoding** usado para especificar la asignación de estados en una FSM.

- **Type** Estados is (Inicio, Uno, dos, Tres, Cuatro);

Attribute syn_enum_encoding : string;

Attribute syn_enum_encoding of Estados : type is "000 001 011 111 110";

-- Se deben poner estos atributos después de la declaración de tipo

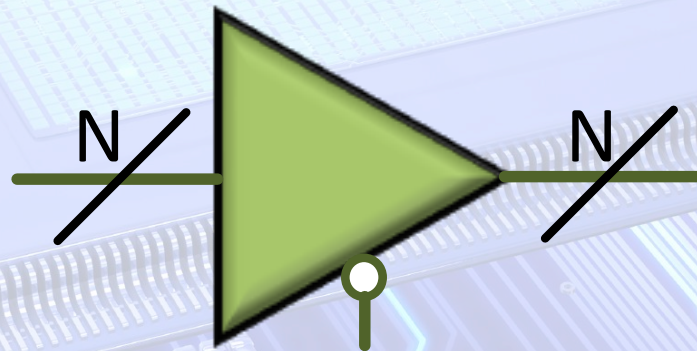
Signal Estado_actual, Estado_siguiente : Estados;

-- Se debe poner estos atributos; de lo contrario no respeta la codificación

9. Problemas

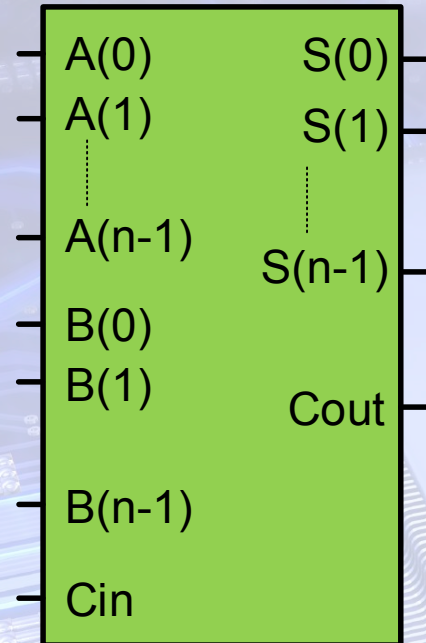
❖ Diseño 1. Buffer de tres estados de n bits.

- Describir en VHDL el buffer de tres estados de n bits de la figura. **Din** son las entradas del buffer, **Dout** las patillas de salida y **OE** la entrada de habilitación de las salidas.



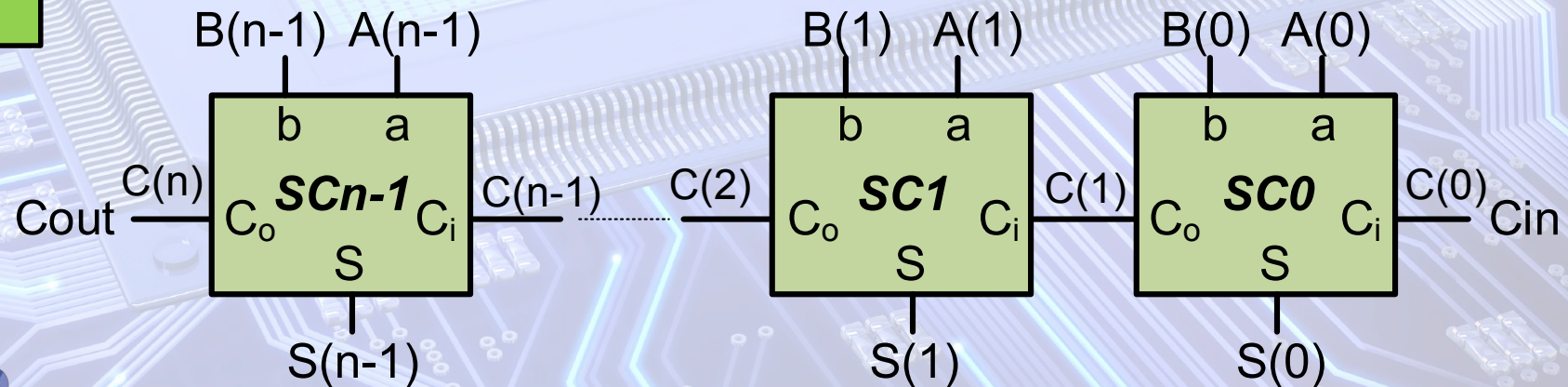
9. Problemas

Sumador binario



❖ Diseño 2. Sumador binario de n bits.

- Describir en VHDL mediante un bucle for loop el sumador binario de la figura.
- Se realizará a partir de n sumadores completos de 1 bit con propagación de acarreo en serie, tal y como se vio en la asignatura FEC, el cual se muestra en la figura adjunta.

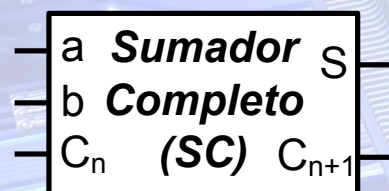


9. Problemas

❖ Diseño 2. Sumador binario de n bits (continuación)

- Funciones lógicas de las salidas del sumador completo.

Representación a nivel de bloque



	bC_n			
a	00	01	11	10
0			1	
1		1	1	1

$$C_{n+1} = ab + aC_n + bC_n$$

Tabla de verdad

a	b	C_n	C_{n+1}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

	bC_n			
a	00	01	11	10
0		1		1
1	1		1	

$$S = a\bar{b}\bar{C}_n + abC_n + \bar{a}bC_n + \bar{a}b\bar{C}_n$$

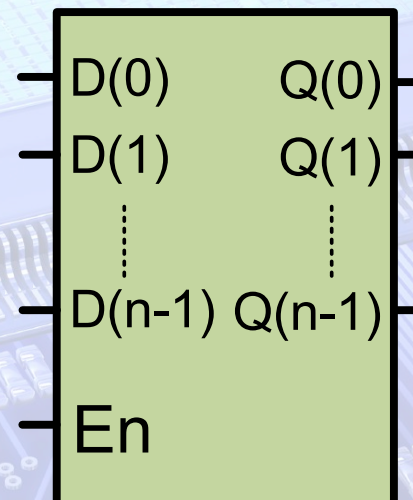
$$= a \oplus b \oplus C_n$$

9. Problemas

❖ Diseño 3. Latch transparente de n bits.

- Describir en VHDL el latch D transparente de n bits de la figura.
 - ✓ **En** es la entrada de habilitación.
 - ✓ Es activa a nivel lógico alto

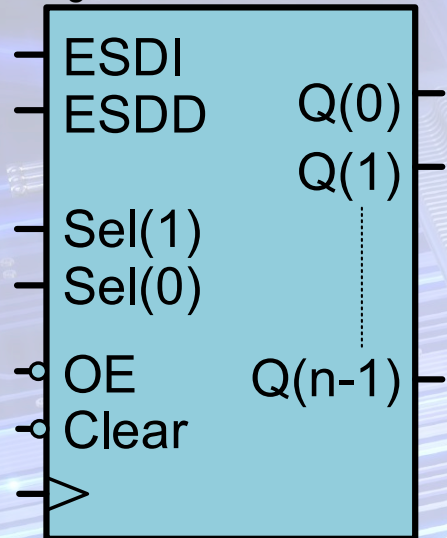
Latch D de n bits



9. Problemas

❖ Diseño 4. Registro universal de n bits.

Registro universal de n bits



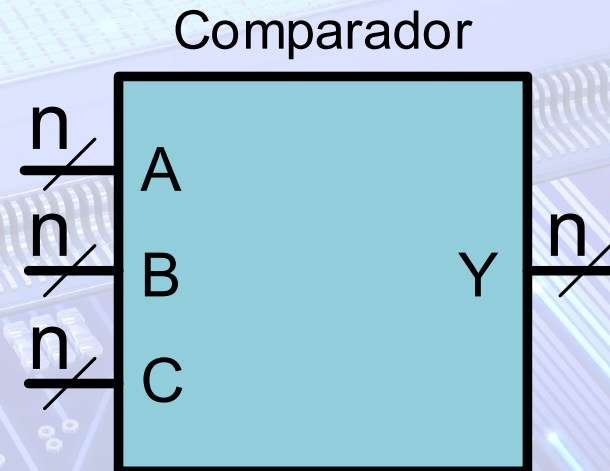
- Describir en VHDL el registro universal de n bits con patillas de datos bidireccionales de la figura.
 - ✓ Clear es la entrada de puesta a cero asíncrona.
 - ✓ OE es la entrada de habilitación de las salidas de datos.
 - ✓ Se tendrá en cuenta que las patillas de datos se conectan a un bus, por lo que la entrada y salida de datos del registro se hacen a través de éste.
 - ✓ ESDI es la entrada serie del desplazamiento a la izquierda y ESDD la entrada serie del desplazamiento a la derecha.
 - ✓ Las entradas Sel determinan el modo de funcionamiento del registro en base a la siguiente tabla:

Sel(1)	Sel(0)	Modo de funcionamiento
0	0	Mantiene su contenido
0	1	Desplazamiento a la derecha
1	0	Desplazamiento a la izquierda
1	1	Carga paralela

9. Problemas

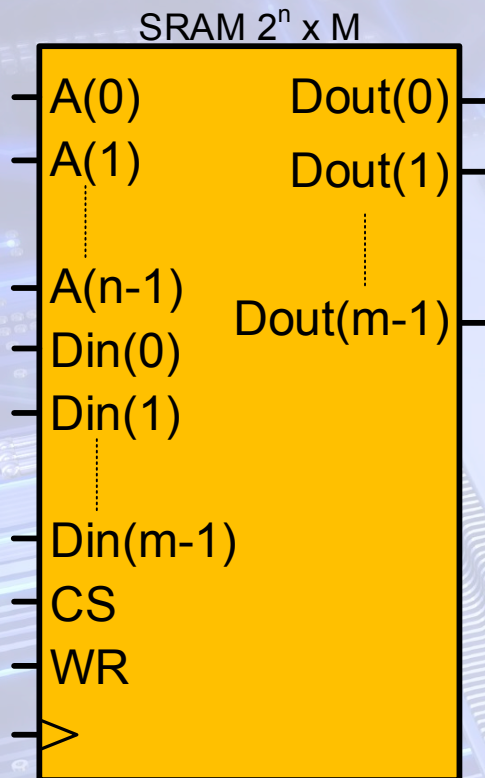
❖ Diseño 5. Comparador de magnitud de n bits.

- Describir en VHDL el comparador de magnitud de la figura.
 - ✓ El comparador pondrá en su salida Y el número de mayor magnitud aplicado en sus tres entradas, A, B y C.
 - ✓ La descripción se realizará para un comparador de n bits.



9. Problemas

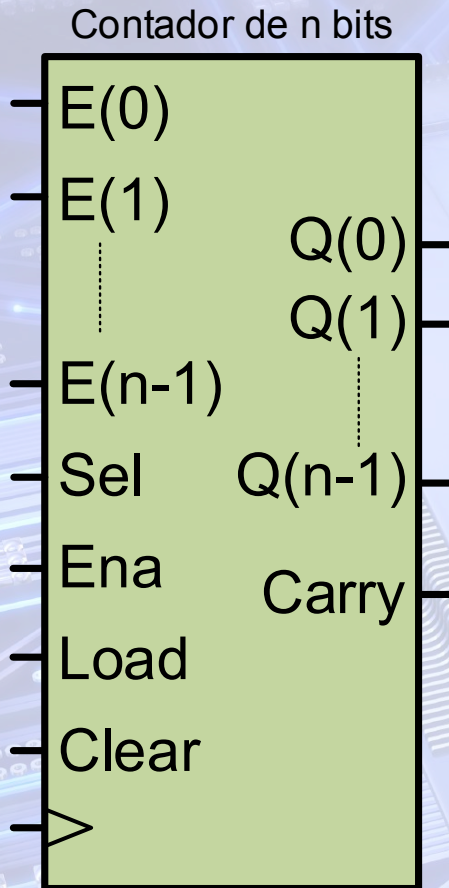
❖ Diseño 6. SRAM de $2^n \times m$



- Describir en VHDL la SRAM (Memoria de Acceso Aleatorio Estática) de la figura.
 - ✓ Se trata de las SRAM típicas que contienen las FPGAs, en las cuales la escritura es síncrona, y tienen patillas independientes para la entrada de datos en la operación de escritura (Din) y para la salida de datos en la operación de lectura (Dout).
 - ✓ La descripción debe de hacerse de forma genérica para una memoria de n bits de direcciones y m bits de tamaño de palabra.
 - ✓ CS (Chip Select) es la entrada de selección de la memoria. Para que se pueda realizar una operación de lectura o escritura, CS tiene que estar activa.
 - ✓ Si se produce un flanco de subida en Clk y WR está activa, se almacena en la palabra seleccionada por las líneas de direcciones (A) el dato de entrada (Din).
 - ✓ Si se activa CS la SRAM pone en sus patillas de datos de salida (Dout) el contenido de la palabra seleccionada por las líneas de direcciones A.
 - ✓ Si CS está desactiva las patillas de datos están a cero.

9. Problemas

❖ Diseño 7. Contador binario reversible de n bits



- Describir en VHDL el contador binario reversible de n bits con carga paralela.
 - ✓ Clear es la entrada de puesta a cero asíncrona.
 - ✓ Load es la entrada de control para la carga paralela. Si Load se activa y Clear está desactiva el contador realiza la carga síncrona, es decir la cuenta (salida de contador) es igual a las entradas de datos de la carga paralela (E).
 - ✓ Ena es la entrada de habilitación de la cuenta.
 - ✓ Por tanto, el contador actualizará su cuenta sólo si Load está desactiva y Ena activa.
 - ✓ Sel indica la dirección de la cuenta, de forma que si se pone a 0 se selecciona la cuenta ascendente.
 - ✓ Carry es la salida de acarreo, que se activará sólo cuando esté la cuenta habilitada y el contador en su cuenta final, que dependerá de la dirección de la cuenta seleccionada por Sel (ascendente o descendente).